step 1

1.树是由根节点引出的n个节点，每个节点都可以指向下一个节点的一种数据结构（不能由子节点指向高级节点）。二叉树则是指每个节点最多有两个分叉（孩子）的特殊树。

2.二叉树可以由数组直接存储或使用指针将每个节点连接起来。二叉树的特点在于它特有的数学性质：

- 深度为 k的二叉树最多有 2^k−1 个结点。
- 叶子结点数 n0与度为 2 的结点数 n2 满足：n0=n2+1

完全二叉树情况：由于每一层的节点数（填满的情况）都是上一层的两倍

- 若从1开始编号，则结点 i的左孩子是 2i，右孩子是 2i+1，父结点是 [i/2].
- h=⌊log2n⌋+1也可以写成：h=⌈log2(n+1)⌉。

3.关于**顺序表**的思考。如何初始化？要注意，树的变量名并不能在函数内部**首次**出现。所以先在main中声明SeqBiTree，再在init_tree中通过循环遍历。根节点的返回值是*bool*,暗示我们检查tree是否为空。并且在数组1处创建根节点方便数学计算。左右孩子的创建关键是要检查是否越界，防止错误写入，然后节点数组下标自然可以根据二叉树的性质由父节点决定。层序遍历的关键是如何找到2^n,我这里设了int n=1，每次换行时让n=2*n，成立。

step 2

函数见代码中。

为什么用for无法打印二叉树？我感觉因为for是一条线的不断往后递推，无法实现二叉树中选择的过程。

step 3

1.改变*printf*的位置意味着让其出现在不同的递归所返回的区域（或者尚未递归就开始打印（前序）），因此会改变输出结果。以**前序遍历**为例，它先打印出根节点，然后进入左节点开展下一过程，打印下一个根节点直到下一个左节点为**NULL**，进入右节点重新开始，直到**右节点**进行完后往**根节点**返回。照此过程打印出整个树。

2.循环我能很明显的知道它每一步要干什么，可以自己推演。但刚开始接触递归时，它给我最大的冲击是有特别多“岔路”，让我难以模拟（这也是为什么我会打上批注）。做了这么多递归题，我总结出两点：**1.递归函数的实现一定要将此函数拆为更小的可执行单元，重新调用，2.递归的出口必须与递归函数无关。**虽然很浅薄，但毕竟我自己想出来时挺开心的，就写这了。总而言之，递归像分，让代码接近问题的结构；循环像合，让代码类似机器的执行。

3.int参数不需要指针是因为我只需要这两个值的大小，并不需要一块特定的内存来存储或改变这个值。

注释写在源代码中。